COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	08/07/2026	**DATE SUBMITTED:**	08/07/2026
NAME:	Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5.1 Queues

1. PROCEDURE OUTPUT

What procedures did you perform?

```cpp
#include <iostream>
#include<queue>

void displayAll(std::queue<char> copyQ);

int main(){

    //create an object:
    std::queue<char> myQ;



    //use the enqueue operation
    myQ.push('S');
    myQ.push('E');
    myQ.push('T');
    myQ.push('H');
    std::cout<<"The current from is: "<<myQ.front()<<std::endl;
    std::cout<<"The current back is: "<<myQ.back()<<std::endl;
    displayAll(myQ);
    //use the dequeue operation
    myQ.pop();
    displayAll(myQ);
    myQ.pop();
    displayAll(myQ);
    myQ.pop();
    displayAll(myQ);
    myQ.pop();

    //check the queue if it is empty
    std::cout<<"Is the queue empty?"<<myQ.empty()<<std::endl;
    return 0;
}
```

Figure 1. Code Snippet of main.cpp ILO A

```
34  //note only use the member functions of the queue STL
35  void displayAll(std::queue<char> copyQ){
36      //create a copy of the queue
37      std::queue<char> temp(copyQ);
38      // loop until empty
39      while(!temp.empty())
40      {
41          // display the front
42          std::cout<<temp.front()<< " ";
43          // dequeue the front
44          temp.pop();
45      }
46      // add a new line
47      std::cout << std::endl;
48  }
```

Figure 2. Code Snippet of main.cpp ILO A

```
The current from is: S
The current back is: H
S E T H
E T H
T H
H
Is the queue empty?1
```

Figure 3. Snippet of output ILO A

```cpp
#include <iostream>
#include "queue_ll.h"
int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    std::cout<<"Testing the enqueue operator: \n"<<std::endl;
    enqueue('S', &front, &back);
    std::cout<<"Front: "<<front->data << " "<<"Back: "<<back->data<<std::endl;
    enqueue('E', &front, &back);
    std::cout<<"Front: "<<front->data << " "<<"Back: "<<back->data<<std::endl;
    enqueue('T', &front, &back);
    std::cout<<"Front: "<<front->data << " "<<"Back: "<<back->data<<std::endl;
    enqueue('H', &front, &back);
    std::cout<<"Front: "<<front->data << " "<<"Back: "<<back->data<<std::endl;

    display_All(&front, &back);

    std::cout<<"Testing the dequeue operator: \n"<<std::endl;
    dequeue(&front, &back);
    std::cout<<"Front: "<<front->data << " "<<"Back: "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<"Front: "<<front->data << " "<<"Back: "<<back->data<<std::endl;
    dequeue(&front, &back);
    dequeue(&front, &back);
    dequeue(&front, &back);

    isEmpty(&front, &back);


    return 0;
}
```

Figure 4. Code Snippet of main.cpp ILOB

main.cpp ...\ILOA | main.cpp Hoa3p1 | main.cpp ...\

HOA5p1 > ILOB > C queue_ll.h > ...

```cpp
#ifndef QUEUE_LL_H
#define QUEUE_LL_H

//create a class node

template<typename T>
class qNode{
    public:
        T data;
        qNode* next;
};

// new node creator

template<typename T>
qNode<T>* new_node(T newData){
    //allocate a space for the new node
    qNode<T>* newNode = new qNode<T>;
    //store the newdata to the newNode
    newNode->data = newData;
    //point the newNode to the next
    newNode->next = nullptr;

    return newNode;
}

//
```

Figure 5. Code Snippet of queue_ll.h

```
27  //enqueue
28  template<typename T>
29  void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
30      //create a new node
31      qNode<T>* newNode = new_node(newData);
32
33      //check if queue is empty
34      if((*frontPtr) == nullptr && (*backPtr)==nullptr){
35          (*frontPtr) = newNode;
36          (*backPtr) = newNode;
37          return;
38      }
39      // if not then insert at the back
40      (*backPtr)->next = newNode; // Links the current back to the new node
41      (*backPtr) = newNode; // assigns the backptr to the new node
42
43  }
```

Figure 6. Code Snippet of enqueue function

```cpp
//dequeue
template<typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //check if the queue is empty
    if((*frontPtr)==nullptr && (*backPtr)==nullptr){
        std::cout<<"The queue is empty"<<std::endl;
        return;
    }
    //create a temporary variable to store the new node to be deleted
    qNode<T>* deleteTemp = nullptr;

    //assign the current front to the deleteTemp

    deleteTemp = (*frontPtr);

    //check if the queue has only 1 item
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        //means only 1 item is inside the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;
    }

    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;
    //delete newNode
    delete deleteTemp;
}
```

Figure 7. Code Snippet of dequeue function

```
74  //display all
75  template <typename T>
76  void display_All(qNode<T>** frontPtr, qNode<T>** backPtr){
77      //Error catching
78      if((*frontPtr)==nullptr && (*backPtr)==nullptr){
79          std::cout<<"The queue is empty"<<std::endl;
80          return;
81      }
82      std::cout<<"Displaying the stack: \n";
83      qNode<T>* temp = *frontPtr;
84      while(temp !=nullptr){
85          std::cout<< temp->data << "\n";
86          temp = temp->next;
87      }
88      std::cout<<"\n";
89
90  }
91  //isEmpty
92  template<typename T>
93  bool isEmpty(qNode<T>** frontPtr, qNode<T>** backPtr){
94      return *frontPtr == nullptr;
95  }
96
97  #endif
98
```

Figure 8. Code Snippet of display_All and isEmpty

```
Testing the enqueue operator:

Front: S Back: S
Front: S Back: E
Front: S Back: T
Front: S Back: H
Displaying the stack:

Front: S Back: S
Front: S Back: E
Front: S Back: T
Front: S Back: H
Displaying the stack:
Front: S Back: T
Front: S Back: H
Displaying the stack:
Displaying the stack:
S
E
T
S
E
T
E
T
T
H

Testing the dequeue operator:

Front: E Back: H
Front: T Back: H
The queue is empty
```

Figure 9. Snippet of Output ILO B.1

CPE21S2-REALUBIT-ACTIVITIES > HOA5p1 > ILOB.2 > C queueArr.h > ...

```
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>

//Array-Based Circular Queue
template<typename T>
class queueArr{
    private:
        //pointer to dynamically allocate the array
        T* q_array;
        //maximum number of elements in the queue
        size_t q_capacity;
        //current number of elements in the queue
        size_t  q_size;
        //Index of the front element in the queue
        int q_front;
        //Index of the back element in the queue
        int q_back;
    public:
        //constructor
        queueArr(size_t capacity = 10);
        //copy constructor
        queueArr(const queueArr& other);
        //copy assignment operator
        queueArr& operator=(const queueArr& other);
        //destructor
        ~queueArr();

        //queue operations:
        bool Empty();
        bool Full();
        size_t Size();
        void Clear();
        T Front();
        T Back();
        void Enqueue(T value);
        T Dequeue();

};

```

Figure 10. Code Snippet of ILO B.2p1

```
41  //constructor
42  template<typename T>
43  queueArr<T>::queueArr(size_t capacity){
44      //initialize the queue capacity
45      q_capacity = capacity;
46      //initialize the queue size, front, and back
47      q_size = 0;
48      q_front = 0;
49      q_back = -1;
50      q_array = new T[q_capacity];
51  }
52
```

Figure 11. Code Snippet of ILO B.2 Constructor

```
53  //copy constructor
54  template <typename T>
55  queueArr<T>::queueArr(const queueArr& copy){
56      q_capacity = copy.q_capacity;
57      q_size = copy.q_size;
58      q_front = copy.q_front;
59      q_back = copy.q_back;
60      q_array = new T[q_capacity];
61      for(size_t i = 0; i < q_size; i++){
62          q_array[(q_front + i) % q_capacity] = copy.q_array[(copy.q_front + i) % copy.q_capacity];
63      }
64  }
65
```

Figure 12. Code Snippet of ILO B.2 Copy Constructor

```
66  //copy assignment operator
67  template <typename T>
68  queueArr<T>& queueArr<T>::operator=(const queueArr& copy){
69      if(this == &copy){
70          return *this;
71      }
72      delete[] q_array;
73      q_capacity = copy.q_capacity;
74      q_size = copy.q_size;
75      q_front = copy.q_front;
76      q_back = copy.q_back;
77      q_array = new T[q_capacity];
78      for(size_t i = 0; i < q_size; i++){
79          q_array[(q_front + i) % q_capacity] = copy.q_array[(copy.q_front + i) % copy.q_capacity];
80      }
81      return *this;
82  }
83
```

Figure 13. Code Snippet of ILO B.2 Copy Assignment Operator

```
84  //destructor
85  template<typename T>
86  queueArr<T>::~queueArr(){
87      delete[] q_array;
88  }
```

Figure 14. Code Snippet of ILO B.2 Destructor

```
89
90  //Empty()
91  template<typename T>
92  bool queueArr<T>::Empty(){
93      //check if the queue size is empty
94      return q_size == 0;
95  }
96
97  //Full()
98  template<typename T>
99  bool queueArr<T>::Full(){
100     //check if the queue size if full
101     return q_size == q_capacity;
102 }
103
104 //find the current size of the array
105 template<typename T>
106 size_t queueArr<T>::Size(){
107     return q_size;
108 }
109
110 //front
111 template<typename T>
112 T queueArr<T>::Front(){
113     //Check if the queue is empty
114     if(Empty()){
115         std::cout<<"Queue is empty"<<std::endl;
116         return T{};
117     }
118     //return to the front
119     return q_array[q_front];
120 }
121
122 //back
123 template<typename T>
124 T queueArr<T>::Back(){
125     //Check if the queue is empty
126     if(Empty()){
127         std::cout<<"Queue is empty"<<std::endl;
128         return T{};
129     }
130     //return to the back
131     return q_array[q_back];
132 }
133
```

Figure 15. Code Snippet of ILO B.2 Empty, Full, Size, Front, Back

```cpp
//clear:
template<typename T>
void queueArr<T>::Clear(){
    //reset the q_size, q_front, q_back
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

//Enqueue
template <typename T>
void queueArr<T>::Enqueue(T value){
    //Check if the queue is full
    if(Full()){
        std::cout<<"Queue is full"<<std::endl;
        return;
    }
    //move q_back circularly
    q_back = (q_back + 1) % q_capacity;
    //store the value to the back
    q_array[q_back] = value;
    //increment the q_size
    q_size++;

}

//Dequeue
template <typename T>
T queueArr<T>::Dequeue(){
    //Check if the queue is empty
    if(Empty()){
        std::cout<<"Queue is empty"<<std::endl;
        return T{};
    }
    //create a temporary variable to store the current front
    T temp = q_array[q_front];
    //move q_front circularly
    q_front = (q_front + 1) % q_capacity;
    //decrement the q_size
    q_size--;
    //reset the indexes if the queue becomes empty
    if(Empty()){
        q_front = 0;
        q_back = -1;
    }
    //return the temporary variable
    return temp;
}

#endif
```

Figure 16. Code Snippet of ILO B.2 Clear, Enqueue, Dequeue Functions

main.cpp queueArr.h

CPE21S2-REALUBIT-ACTIVITIES > HOA5p1 > ILOB.2 > main.cpp > main()

```cpp
#include <iostream>
#include "queueArr.h"

int main(){

    //creating an object
    queueArr<int> q(5);

    //enqueue 1, 2, 3, 4, 5, 6
    q.Enqueue(1);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;

    q.Enqueue(2);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;

    q.Enqueue(3);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;

    q.Enqueue(4);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;

    q.Enqueue(5);
    std::cout<<"Front: "<<q.Front()<<std::endl;
    std::cout<<"Back: "<<q.Back()<<std::endl;
    std::cout<<"Size: "<<q.Size()<<std::endl;
    q.Enqueue(6);

    std::cout<<"Removed: "<<q.Dequeue()<<std::endl;
    std::cout<<"Removed: "<<q.Dequeue()<<std::endl;

    q.Clear();
    std::cout<<"Is the Queue Empty? "<<(q.Empty()?"Yes":"No")<<std::endl;

    return 0;
}
```

Figure 17. Code Snippet of main.cpp for ILO B.2

Starting with the copy constructor, I was able to achieve the solution shown in Figure 12 by basing it on the given algorithm inside the PDF file. But of course, that was not the complete solution itself since I still had the parameters and lines 61 to 63. For the parameters, luckily enough, I went back to the previous activities that were previously done and just modifying it to fit this piece of code. Lines 61 to 63 specifically are for looping the array in order to copy each element inside the array. Meanwhile, on the copy assignment operator, I was able to achieve that conclusion after some research due to being slightly confused on how to implement the self-assignment without anything actually breaking within the actual program itself.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

printer.h | main.cpp

CPE21S2-REALUBIT-ACTIVITIES > HOA5p1 > Supplementary Activity > printer.h > Printer > frontPtr

```
1   #ifndef PRINTER_H
2   #define PRINTER_H
3   #include "queue_ll.h"
4   #include <string>
5   #include <iostream>
6
7   class Job{
8       public:
9           int jobId;
10          std::string name;
11          int numPages;
12
13          Job(int id, std::string n, int pages): jobId(id), name(n), numPages(pages) {}
14  };
15
16  class Printer{
17      private:
18          qNode<Job>* frontPtr;
19          qNode<Job>* backPtr;
20      public:
21          Printer(): frontPtr(nullptr), backPtr(nullptr) {}
22          void addJob(Job newjob);
23          void processAll();
24          void displayQueue();
25          bool isEmpty();
26  };
27
```

Figure 1. Code Snippet of ILO Cp1

```
28  inline bool Printer::isEmpty(){
29      return (frontPtr == nullptr && backPtr == nullptr);
30  }
31
32  inline void Printer::addJob(Job newjob){
33      enqueue(newjob, &frontPtr, &backPtr);
34  }
35
36  inline void Printer::processAll(){
37      while(!isEmpty()){
38          Job currentJob = frontPtr->data;
39          std::cout << "Job ID: " << currentJob.jobId << ", Name: " << currentJob.name << ", Pages: " << currentJob.numPages << std::endl;
40          dequeue(&frontPtr, &backPtr);
41      }
42  }
43
44  inline void Printer::displayQueue(){
45      qNode<Job>* current = frontPtr;
46      while(current != nullptr){
47          Job currentJob = current->data;
48          std::cout << "Job ID: " << currentJob.jobId << ", Name: " << currentJob.name << ", Pages: " << currentJob.numPages << std::endl;
49          current = current->next;
50      }
51  }
52
53  Printer::~Printer(){
54      while(!isEmpty()){
55          dequeue(&frontPtr, &backPtr);
56      }
57  }
58
59  #endif // PRINTER_H
```

Figure 2. Code Snippet of ILO C p.2

printer.h | main.cpp

CPE21S2-REALUBIT-ACTIVITIES > HOA5p1 > Supplementary Activity > main.cp

```
1   #include <iostream>
2   #include "printer.h"
3
4   int main() {
5       Printer printer;
6
7       printer.addJob(Job(1, "Alice", 5));
8       printer.addJob(Job(2, "Bob", 10));
9       printer.addJob(Job(3, "Charlie", 15));
10
11      std::cout << "Current Print Queue:" << std::endl;
12      printer.displayQueue();
13      std::cout << std::endl;
14      std::cout << "Processing all jobs:" << std::endl;
15      printer.processAll();
16      std::cout << std::endl;
17      std::cout<<"Finished Queue: "<<std::endl;
18      printer.displayQueue();
19
20
21
22      return 0;
23  }
```

Figure 3. Code Snippet of main.cpp

```
Current Print Queue:
Job ID: 1, Name: Alice, Pages: 5
Job ID: 2, Name: Bob, Pages: 10
Job ID: 3, Name: Charlie, Pages: 15

Processing all jobs:
Job ID: 1, Name: Alice, Pages: 5
Job ID: 2, Name: Bob, Pages: 10
Job ID: 3, Name: Charlie, Pages: 15

Finished Queue:
```

Figure 4. Code Snippet of Terminal Output

Conclusion

This hands-on activity taught me about the fundamentals of queues. Particularly about how it should be FIFO (First In, First Out) just like in stacks. Following that was the use of modulo for wrapping q_front and q_back while using arrays. Although we primarily used circular arrays for the activity during the first part of ILO B, I thought it would have been much simpler if we used linked lists using pointers and proper handling of different cases.

In ILO A, the actual introduction of queues was during the procedure, due to using the STL and exploring it, which was similar to stacks, where it had push, pop, top, etc. But in queues, we had different terms, namely enqueue and dequeue, which typically had the same function as push and pop. By far, the most complicated during the procedure was part two of ILO B, due to being slightly unfamiliar with the algorithm type rather than being spoon-fed the actual code itself. Other than that, the copy constructor and copy assignment operator needed different ways of handling themselves than I was usually used to. Despite reusing previous code, it took me a considerable number of errors to actually achieve it.

The printer simulation for the supplementary activity or ILO C was slightly unique. This was because, rather than writing the code from scratch (which is considerably better), I decided to reuse code from the procedure, specifically the first part of Part B or linked lists. This was because, rather than using arrays in implementing this activity, I would rather not think about the actual size of the array itself, and how using pointers on linked lists would be easier in constructing a circular queue. While making ILO C, I ran into a specific issue that I had to get help from AI, specifically the use of the ILO B header file in this code, where new_node was making a node before assigning something into it, thus breaking "Job," which had no default constructor. I was able to solve this problem by making the node directly with the final values.

To be frank, I'm neutral in my solution. The process itself was not completely difficult, but it did take quite a lot of time and simple errors that gave me a couple of headaches at first due to not being able to solve them quickly as I had expected. Only to find out, there were easier methods to actually solve the problem itself rather than being stubborn and sticking to that one specific solution. Notably, the biggest change I made in this activity was adding inline in ILO C; this is because of a curiosity and cautiousness of mine regarding being able to use these files in the future where it is included in multiple projects.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly, Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, grammar correction, error assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **For forgotten concepts and recall, I ask Claude AI for simple examples that would allow me to recall the concepts I've forgotten. Meanwhile, for error assistance, I usually submit the error message after trying to figure it out on my own; this is because I usually add more errors than actually fixing the error itself. Apart from that, I do limit it to not give the actual solution itself but point to the cause of the error, and I try to figure it out on my own unless it leaves me stuck on that error for more than 20-30 minutes. For Grammarly AI, it is usually for missing commas and better wording for the sentences that I make.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.